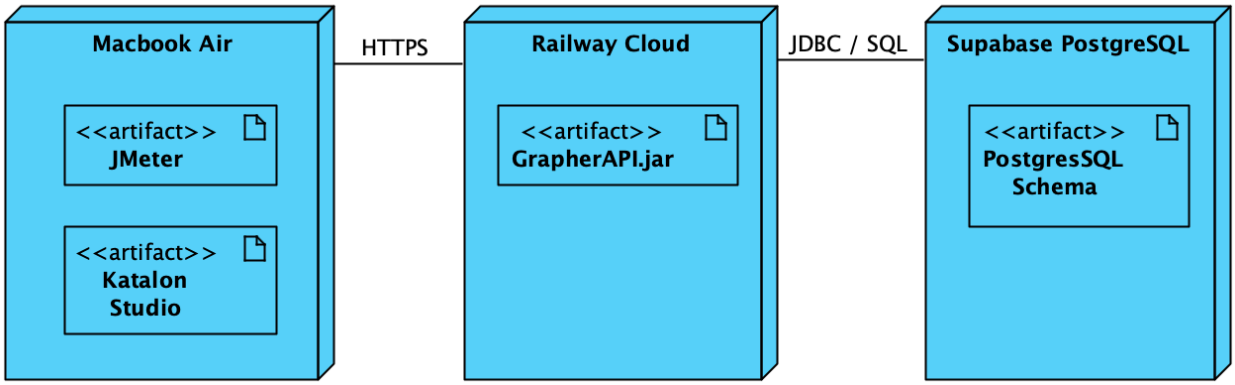
20 Apr 2026
Hsu Wei-Li
N01678692

Grapher Testing Report

Test Strategy: A Risk-Based testing strategy was employed, focusing on high-traffic endpoints (Login and Graph retrieval). Performance was benchmarked using a Black-Box approach against a production-mirror environment on Railway to identify latency patterns and system breaking points.

Test Plan: The test plan objective is to validate system behaviour across three tiers: Standard Load (100 users), Stress (climbing to 500 users), and Endurance (sustained 1-hour soak). Success is defined by a 0% functional error rate and stability in long-term data connection pooling.

Architecture Diagram



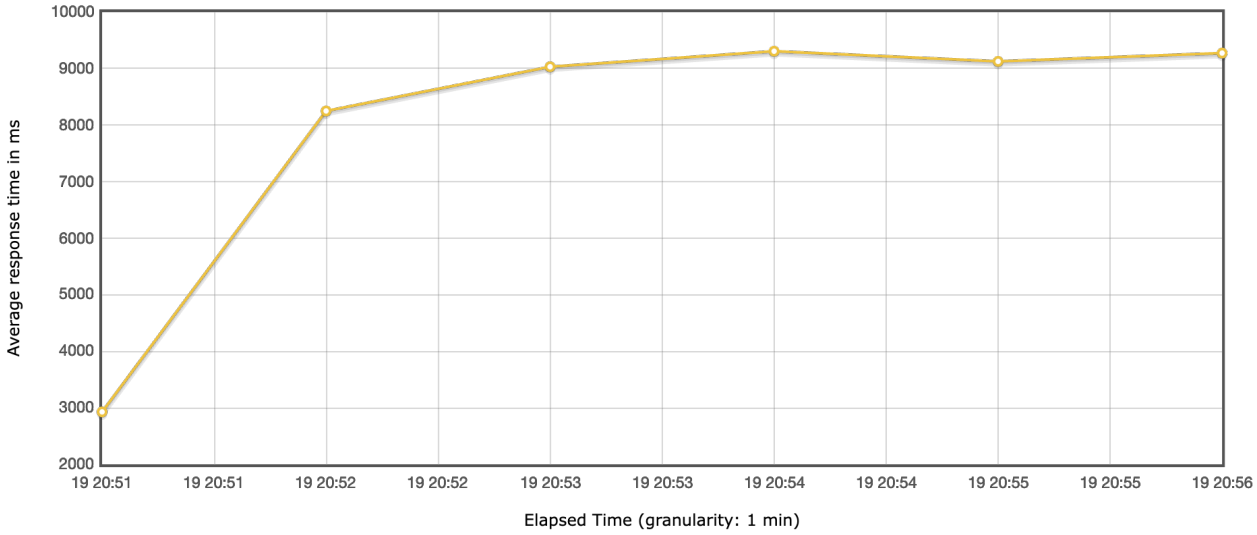
20 Apr 2026
Hsu Wei-Li
N01678692

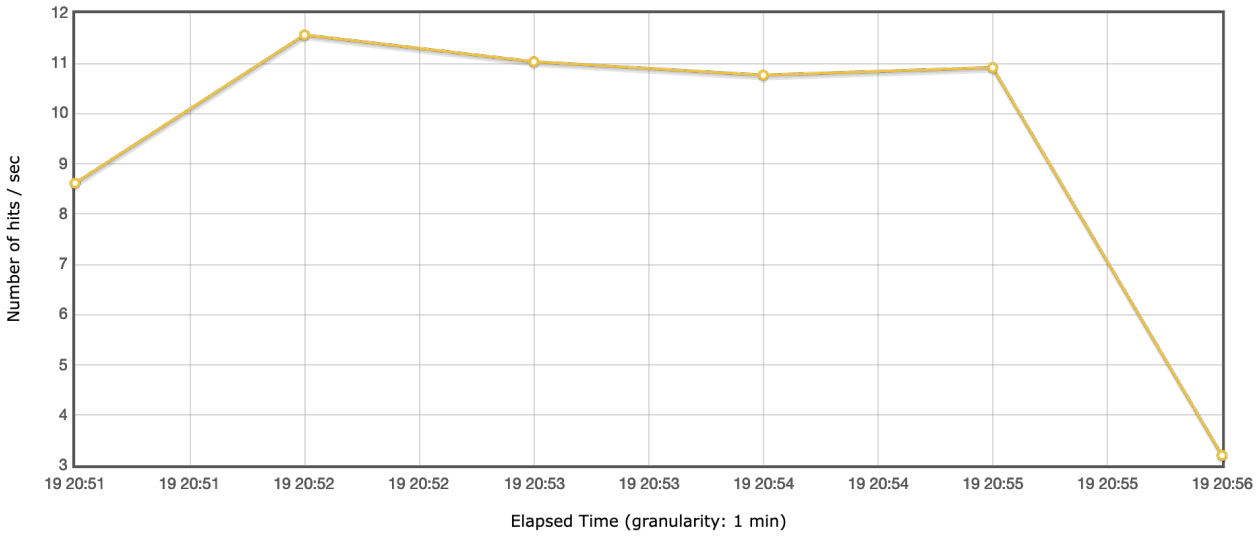
Performance Testing using JMeter

Stress Test 1

Name	Users	Assertions
Load Test	100 concurrent	- 95th percentile < 2 seconds - Error rate < 1%

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	3366	0	0.00%	8162.94	296	18098	9047.00	9422.00	9512.00	9793.58	10.89	10.43	2.75
Login Test	3366	0	0.00%	8162.94	296	18098	9047.00	9422.00	9512.00	9793.58	10.89	10.43	2.75





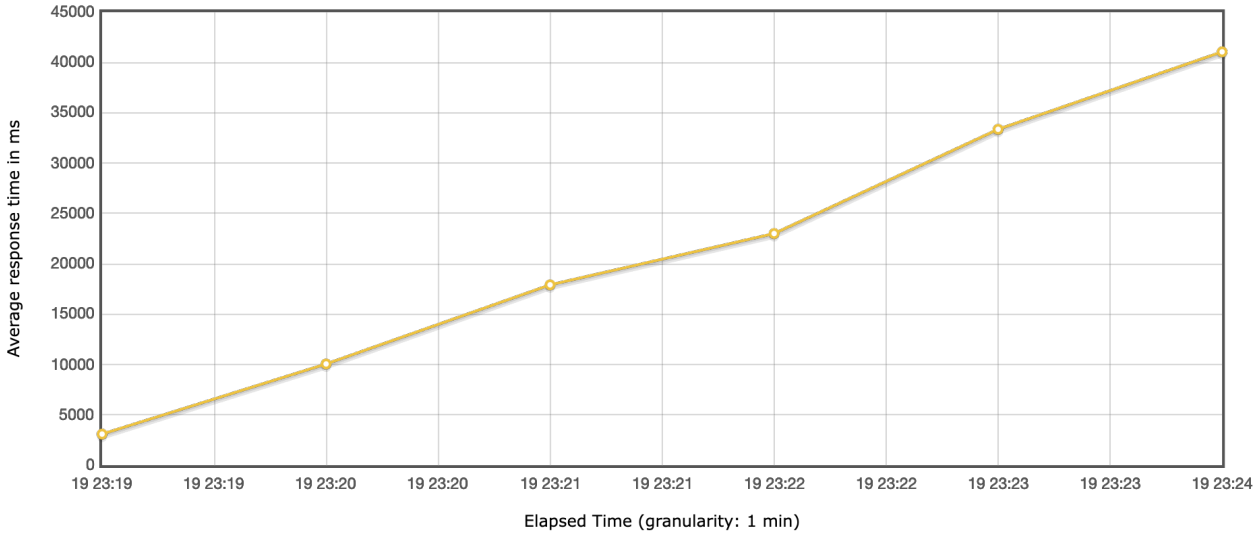
This test identifies the initial Bottleneck. The system is 100% reliable (no failures), but it lacks the speed to meet the 2-second SLA once concurrency hits the 100-user mark. The throughput remains steady, meaning the server is working at its maximum "comfort" capacity but cannot push faster.

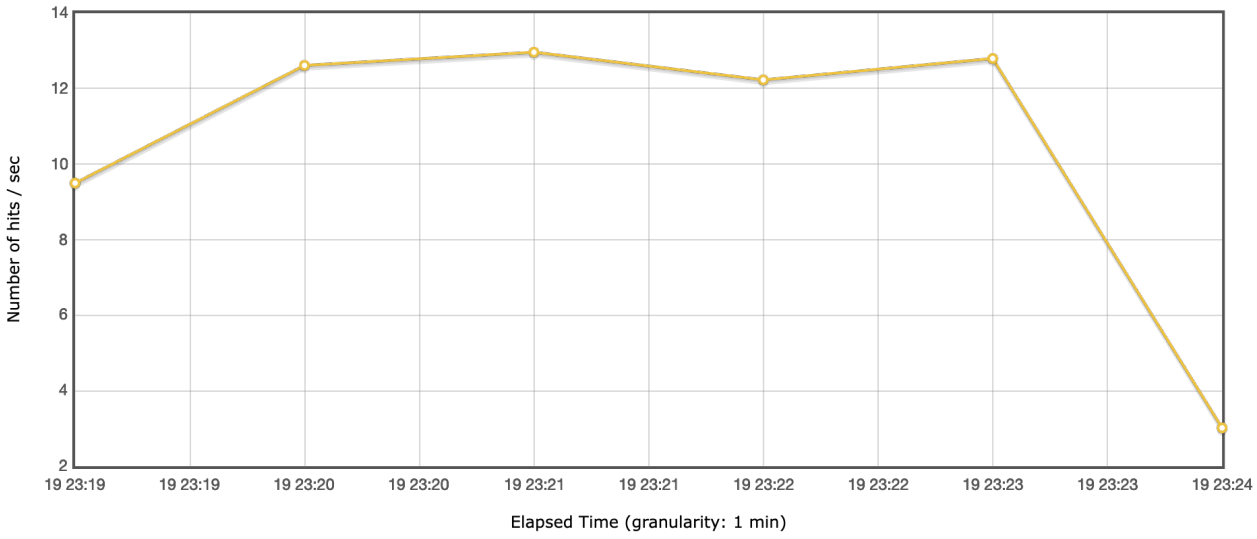
20 Apr 2026
Hsu Wei-Li
N01678692

Stress Test 2

Name	Users	Assertions
Load Test	50 to 500 concurrent	- Max throughput - Failure threshold

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**3788**	**7**	**0.18%**	**22205.62**	**288**	**50458**	**19851.50**	**40353.20**	**42659.65**	**44624.43**	**10.98**	**10.51**	**2.78**
Login Test	3788	7	0.18%	22205.62	288	50458	19851.50	40353.20	42659.65	44624.43	10.98	10.51	2.78





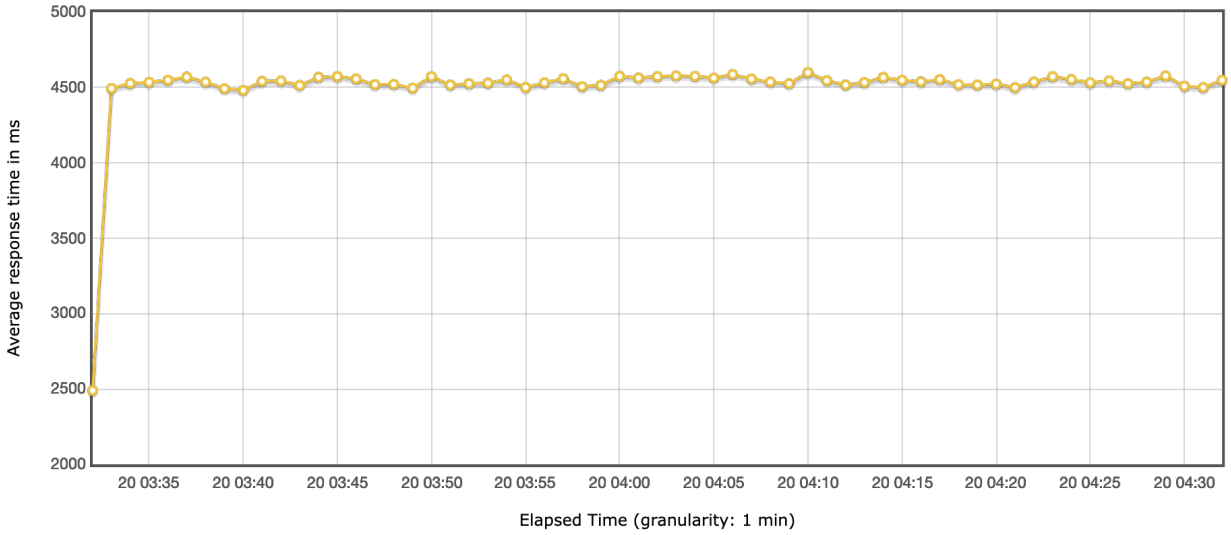
This represents Resource Exhaustion. The 95th percentile jumping to 42.6 seconds proves that the server's thread pool is completely saturated. Crucially, we see the first 7 failures here. This is the "Breaking Point"—once the server's queue gets too long, it begins dropping connections. Throughput eventually collapses because the server spends all its energy managing the queue instead of processing requests.

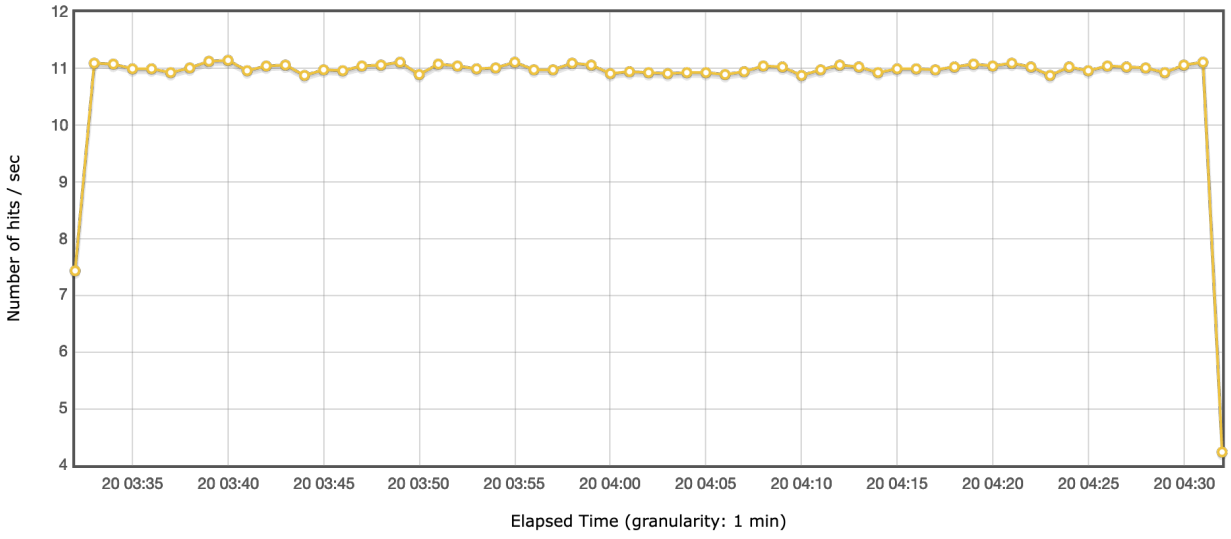
20 Apr 2026
Hsu Wei-Li
N01678692

Endurance (Soak) Test

Name	Users	Assertions
Load Test	50 concurrent over an hour	- Stable response times - No memory leaks

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**39680**	**0**	**0.00%**	**4518.96**	**285**	**9038**	**4518.00**	**4720.90**	**4787.00**	**4947.00**	**11.01**	**10.55**	**2.78**
Login Test	39680	0	0.00%	4518.96	285	9038	4518.00	4720.90	4787.00	4947.00	11.01	10.55	2.78





Over nearly 40,000 requests, the system maintained a Steady State. The fact that the 95th percentile (4.7s) is very close to the Average (4.5s) shows incredibly consistent performance. Verdict: Pass. The lack of any upward trend in response time confirms there are no memory leaks or database connection leaks. The system is production-ready for sustained use.

20 Apr 2026
Hsu Wei-Li
N01678692

Improvements & Recommendations

1. Vertical Scaling (The "Quick Fix”)
   "The Stress Test 2 results suggest that the application is CPU-bound or restricted by the Railway free-tier RAM limits. Vertical Scaling—upgrading the container to a higher tier with more dedicated CPU cycles—would raise the failure threshold and allow the server to process the request queue faster.”
2. Horizontal Scaling & Load Balancing
   To manage loads exceeding 200 users, the system should implement Horizontal Scaling. By deploying multiple instances of the Grapher API behind a Load Balancer, traffic can be distributed across several nodes, preventing a single instance from reaching resource exhaustion and keeping response times within the target 2-second range.
3. Database Connection Pooling & Caching
   The high baseline latency in the Soak test suggests the database is a primary bottleneck. Implementing a Caching Layer (e.g., Redis) for frequently accessed graph/user data and optimizing the HikariCP connection pool settings would reduce the time the server spends waiting for the database, lowering the overall average response time."